Agentic AI for Research

What the people who do this every day have learned, and what it means when your name is on the result

Congyue Cui · **Lucas Sawade** · Princeton Research Computing · October 2026



You just watched it get built from nothing

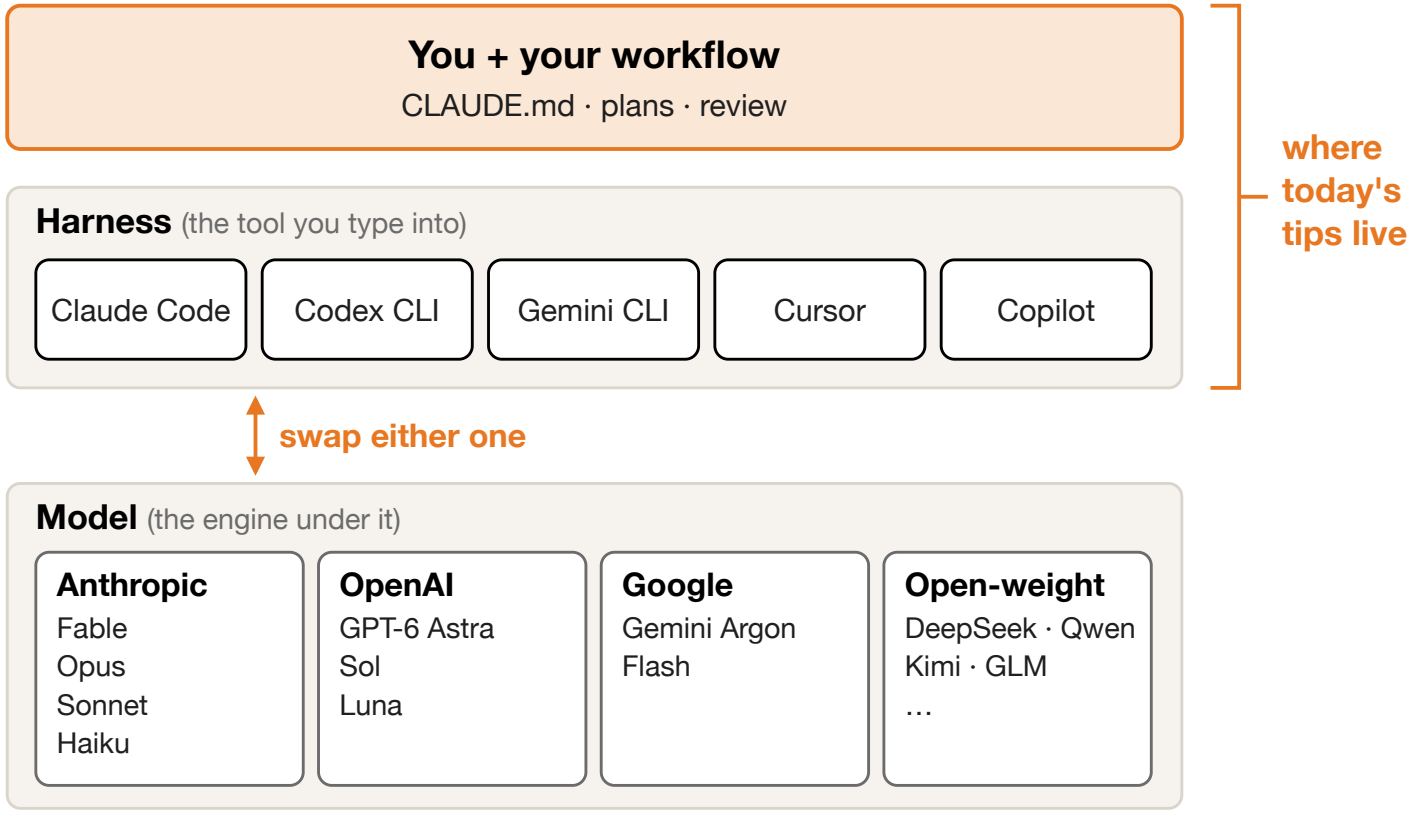
An hour ago · you watched it build → **Now · how to supervise it**

1. The model landscape
2. Tokens, cost and delegation
3. Your name is on it
4. Setting up the workspace
5. Working patterns from your colleagues
6. Starter kit and discussion

Interrupt me. Two of us are up here; the one not speaking is watching for your hand.

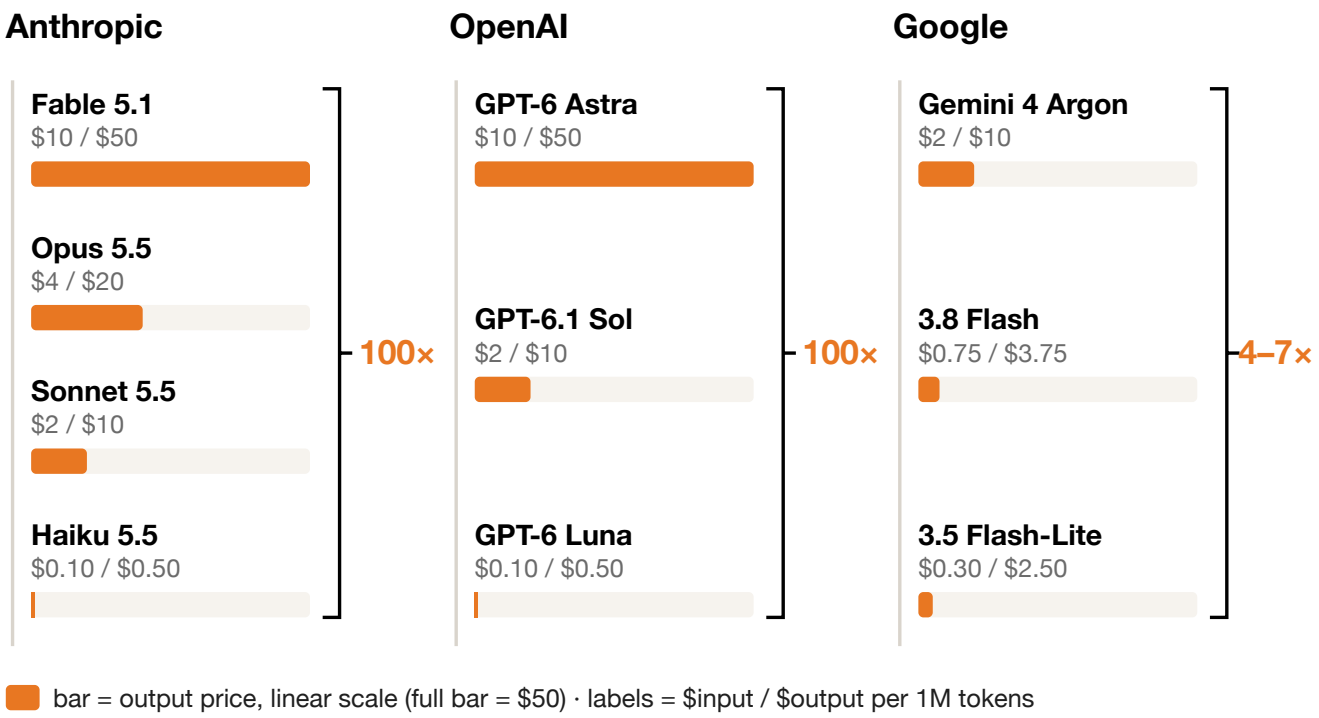
Keep the build in mind. We will point back at it: the retries, the unread diff, the missing CLAUDE.md.

Model ≠ tool: you can swap either one



Most of today's tips are about the top two layers, so they carry over from Claude Code to Codex, Cursor and Gemini CLI.

Families ship as ladders, up to 100× tall

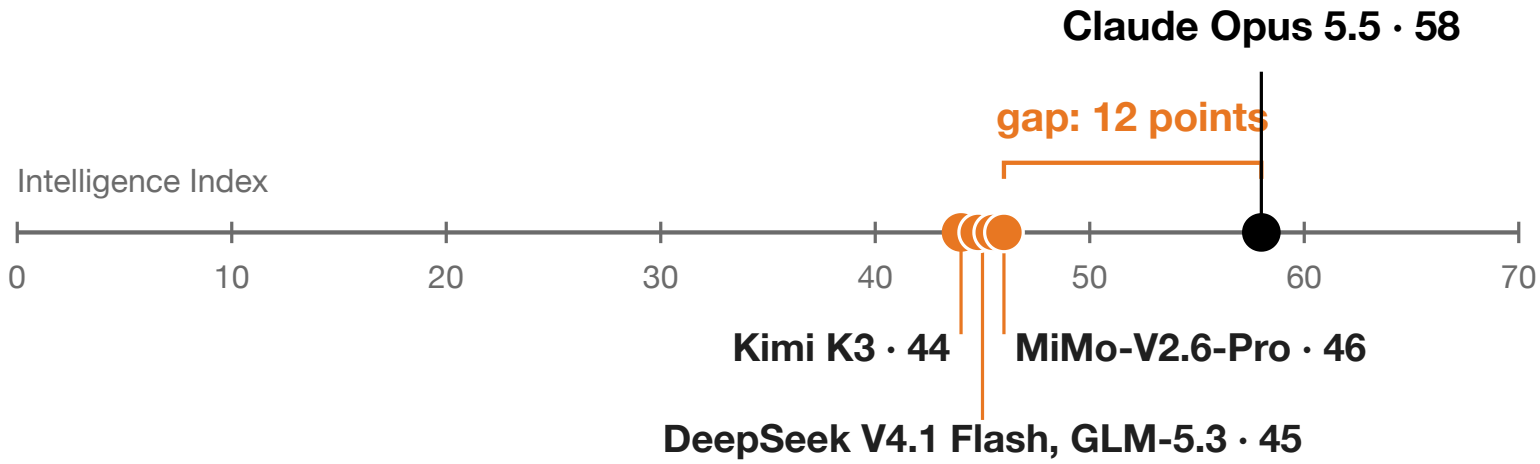


Flagships now think adaptively; you set effort, not on/off.

1M-token context is standard at the frontier.

Prices as of 20 Oct 2026, USD per 1M tokens (input / output); see research/numbers.md. Gemini 4 Argon doubles on 1 Dec 2026.

Open weights trail the frontier by about a dozen points



Why it matters: restricted data stays on the cluster

Why it matters: per-token cost disappears

The trade-off: you run the infrastructure

Artificial Analysis Intelligence Index, 20 Oct 2026; changes monthly, check the live board.

Pick the tier by the job, not the benchmark

Frontier	**Planning, review, hard reasoning** Fable · Astra · Argon
Mid	**Daily work: writing and editing code** Opus · Sonnet · Sol
Small	**Search, grep, test runs, summaries** Haiku · Luna · Flash-Lite

Benchmarks are saturated; use live leaderboards: artificialanalysis.ai, openrouter.ai/rankings.

TIP
use multiple models for same problem

TIP
Adversarial development/brainstorming using Claude vs. Codex

Three tasks. Which tier would you use?

1. Find every call to a deprecated function in a 40,000-line codebase.

2. Design the analysis for a new dataset nobody in the group has worked with.

3. Summarize 20 log files from last night's runs into one paragraph each.

Frontier: plans, reviews, hard reasoning · Mid: daily work · Small: search, test runs, summaries

The ladder, applied

1

Find every call to a deprecated function in a 40,000-line codebase. **SMALL**

grep-shaped: many files, no judgement, a cheap model in a subagent

2

Design the analysis for a new dataset nobody in the group has worked with. **FRONTIER**

one hard reasoning task where a wrong plan costs weeks

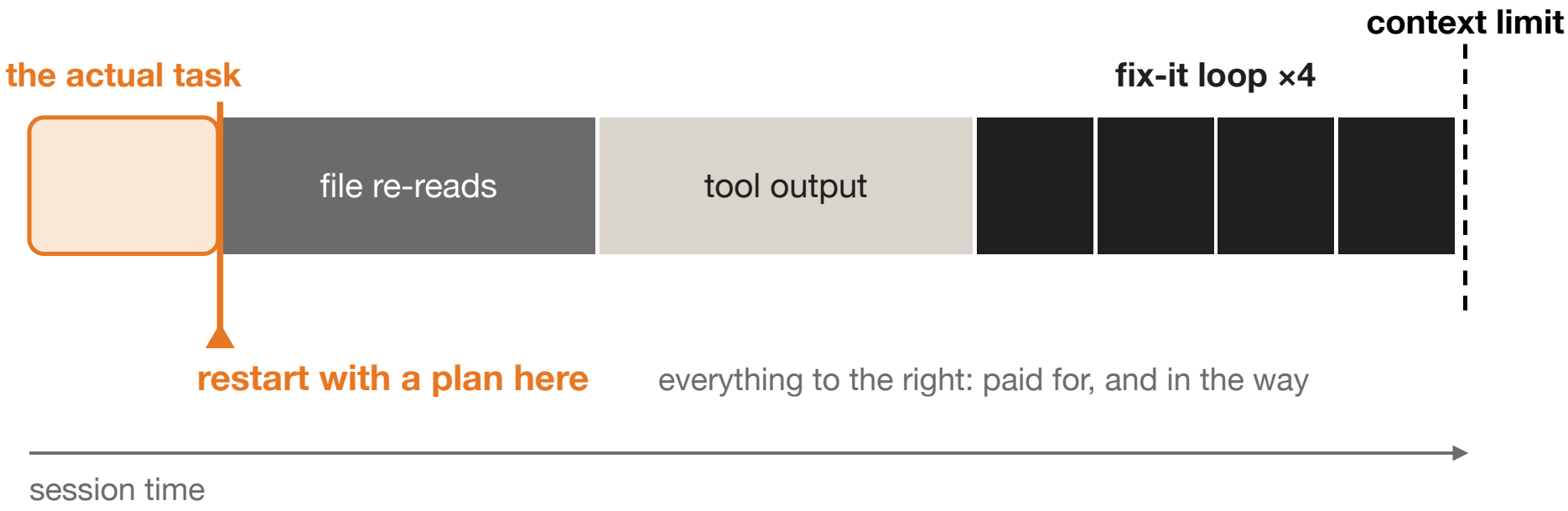
3

Summarize 20 log files from last night's runs into one paragraph each. **SMALL**

20 parallel summaries; the parent only reads the conclusions

Frontier: plans, reviews, hard reasoning · Mid: daily work · Small: search, test runs, summaries

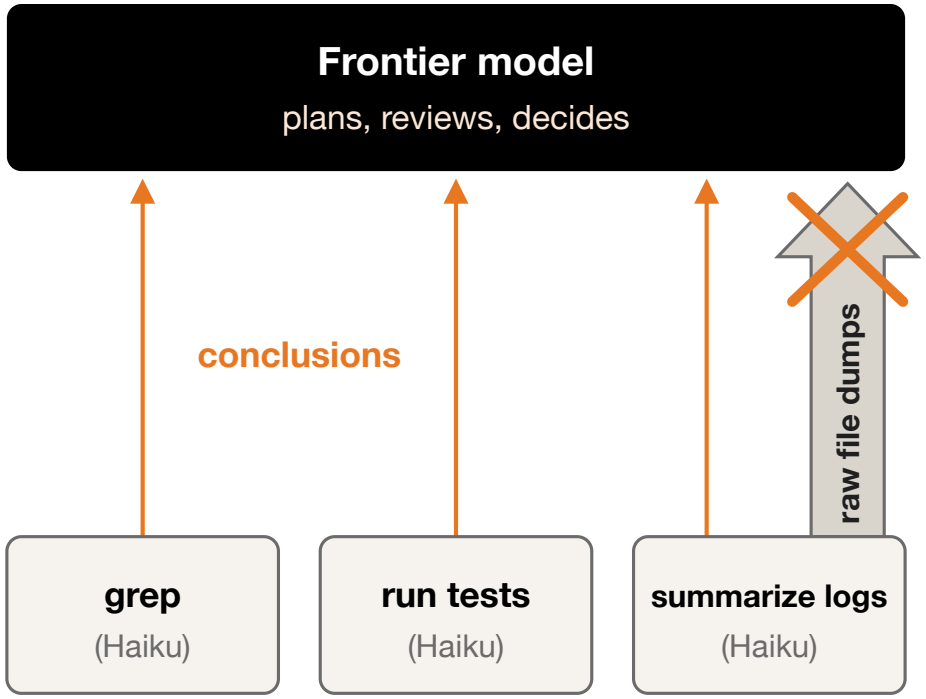
What burns tokens also confuses the agent



The habits that keep the agent sharp also keep the bill down: plan before coding, restart when it drifts.

> **TIP**
> Restarting sessions, it often gets confused with too much context

Big model decides; small models fetch



subagents on a cheap model

```
---
name: test-runner
description: Runs the test suite, reports only failures
model: haiku
tools: Bash, Read, Grep
---
Run `pytest -q`. Do not fix anything.
Report passed/failed counts and, per failure:
test name, assertion message, likely file:line.
No file contents, no full tracebacks.
```

What the parent sees back:

```
47 passed, 2 failed.
test_inversion.py::test_misfit_zero: expected 0.0, got 1.3e-4
(tolerance; misfit.py:88)
test_io.py::test_read_sac: FileNotFoundError data/sample.sac
(fixture missing)
```

This file lives in `.claude/agents/` ; more in section 4.

Four more levers on cost and clarity

Compact or clear. `/compact` summarizes, `/clear` starts fresh; do it between tasks, not mid-task.

Scope CLAUDE.md to what every session needs; everything else goes in a file the agent reads on demand.

Background long jobs and check in; do not sit in a chat window watching tests.

Use the ladder: frontier to plan, mid to build, small to fetch.

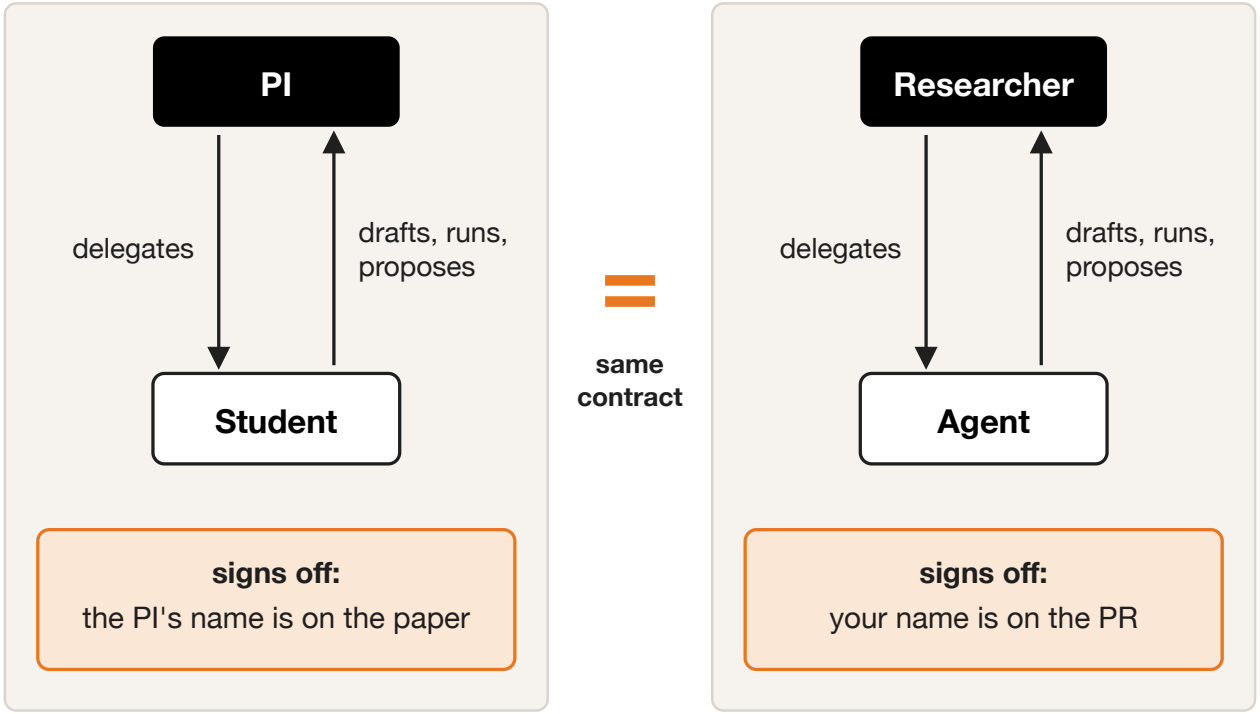
Rough cost per tier, 20 Oct 2026

Tier	$ per 1M output tokens	AA cost per Index task
Fable 5.1	$50	$7.63
Opus 5.5	$20	$5.98
Sonnet 5.5	$10	$5.46
Haiku 5.5	$0.50	$0.21

Cost per task is Artificial Analysis benchmark tasks, not coding sessions. Haiku output price is for prompts up to 100k tokens.

You are the supervisor, not the user

The relationship is closer to PI-and-student than tool-and-user: you delegate, you review, you sign off. Nothing about this is new; authorship has always worked this way. The tool just makes it easier to forget.



It fails in specific ways, so review for them

Failure mode	What it looks like
Plausible-but-wrong code	Runs, returns a number, number is wrong
Hallucinated dependencies and APIs	Imports a package or method that does not exist, or exists with different arguments
Silently changed assumptions	It picked a default (units, date range, NaN handling) you never stated
"Tests pass" that never ran	The report says green; the command was never executed or tested the wrong thing
Confident claims about results	An interpretation stated as fact with no check against data

TIP

Always review the code. It often makes mistakes or assumes things different than your implicit assumptions

TIP

Ask AI to list the problems in the code it generated

A supervisor who knows the typical failure modes reviews better. Slide 5.5 says what to do about each.

Spot the bug 60 s

```
import pandas as pd
COLS = ["station", "time", "temp_c", "rh_pct", "wind_ms", "qc"]
df = pd.read_csv("met_stations.csv", usecols=COLS, parse_dates=["time"])

df = df[df["qc"] == "ok"]
df = df.dropna()                          # remove missing readings
df["year"] = df["time"].dt.year

annual = (df.groupby(["station", "year"])["temp_c"]
            .agg(["mean", "count"])
            .reset_index())
trend = annual.groupby("station")["mean"].apply(lambda s: s.diff().mean())
print(trend.sort_values(ascending=False).head(10))
```

One of these lines silently changes the result. Which?

The bug: `dropna()` throws away good temperatures

```
import pandas as pd
COLS = ["station", "time", "temp_c", "rh_pct", "wind_ms", "qc"]
df = pd.read_csv("met_stations.csv", usecols=COLS, parse_dates=["time"])

df = df[df["qc"] == "ok"]
df = df.dropna()                          # remove missing readings
df["year"] = df["time"].dt.year

annual = (df.groupby(["station", "year"])["temp_c"]
            .agg(["mean", "count"])
            .reset_index())
trend = annual.groupby("station")["mean"].apply(lambda s: s.diff().mean())
print(trend.sort_values(ascending=False).head(10))
```

Would you have caught this in a 400-line PR?

A NaN in humidity or wind deletes a valid temperature. Catch it: test a known case, ask for assumptions.

Open source already expects a supervisor

78%
of project AI policies allow AI-assisted contributions

51%
require disclosure

74%
require a human in the loop

118 AI policies across 1,000 popular GitHub repositories. Hora & Robbes, ICSME 2026, arXiv 2605.16706.

Project	Policy	What it says about the supervisor
Linux kernel	AI-assisted patches allowed; only a human adds Signed-off-by and takes full responsibility; `Assisted-by` tag.	The tool may draft, the human signs.
LLVM	"Human in the loop" (Jan 2026): a human reviews all output; label substantial tool-generated content.	Disclosure is a courtesy to the reviewer whose time you are spending.
GCC	Steering committee (29 Jul 2026) declines "legally significant" LLM-derived code; small, marked changes OK.	When authorship is uncertain, the project would rather not carry the risk.

Fresh: a patch dropping the kernel's Assisted-by tag reached the docs tree on 14 Oct 2026, not yet mainline. Accountability stays.

Research output meets the same standard

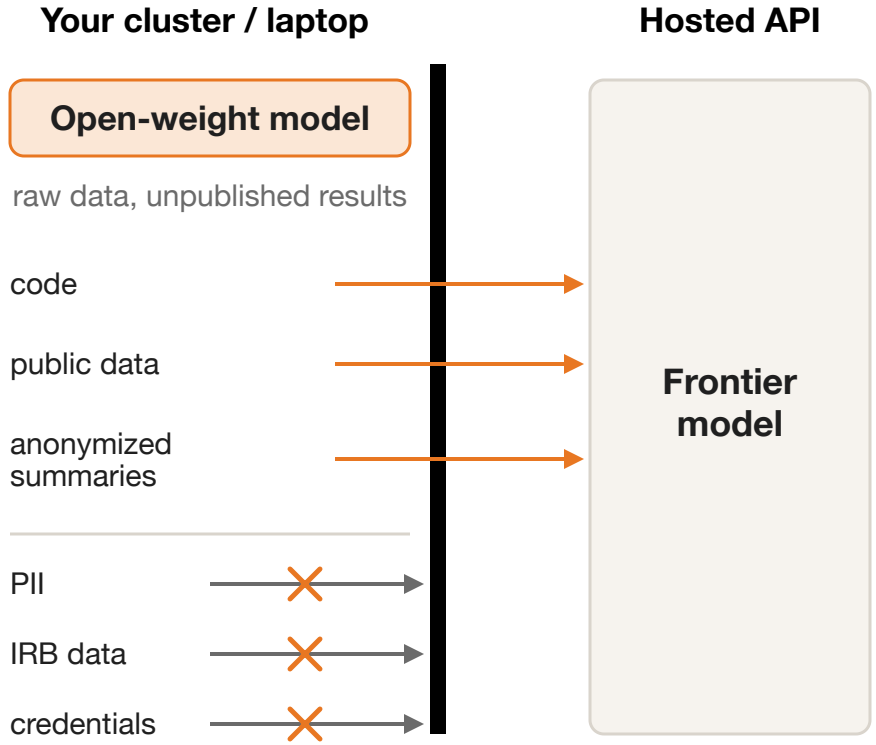
Reproducibility A script you can rerun and review beats an answer you can't.

Disclosure Journals and funders are writing the same rules as the kernel. Check before you submit.

Data Unpublished results, PII, IRB-covered and export-controlled data never go into a hosted model. Use local or open-weight models.

Permissions Sandbox the agent. No live database credentials.

> **TIP**
> Have the AI write code to do the task, not directly do the task



Verification is the hardest job to delegate.

"Verification is the hardest job to delegate."

HANDOFF





RESEARCH COMPUTING

CLAUDE.md: standing instructions for every session

```
# SPECFEM++
C++20 spectral-element seismic wave solver,
portable to CPU and GPU via Kokkos.

## Layout
- `core/specfem/` -- all C++ library code
- `tests/unit-tests/` -- GoogleTest suite
- `docs/` -- Sphinx + Doxygen (RST)

## Critical conventions
- `snake_case` for functions and files
- No `using namespace`; no anonymous namespaces
- `#pragma once`, never `#ifndef`

## Build
cmake --preset <name> && cmake --build build/<name>
ctest --test-dir build/<name>
```

What it is: two lines on the repo, so the agent starts oriented.

How to build and test: the exact commands, so it can check its own work.

Conventions: the rules a newcomer would break first.

What not to touch: generated files, vendored code, legacy readers.

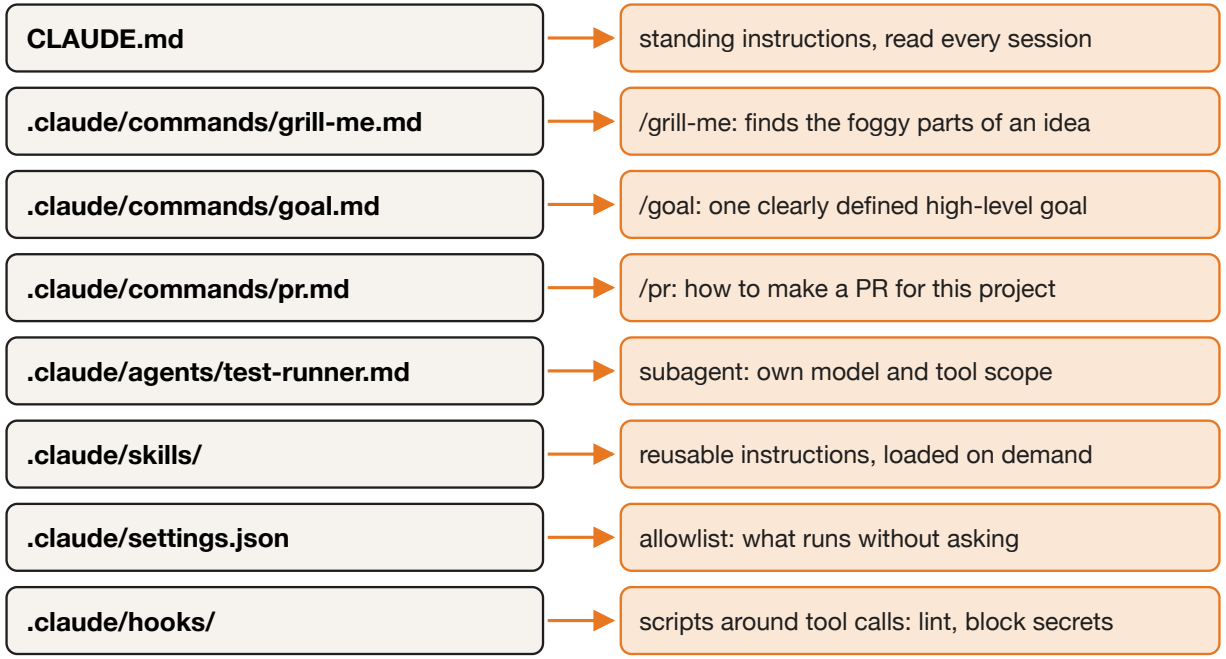
Codex and Cursor read AGENTS.md the same way.

TIP

Clear Agents.md / Claude.md files

The build you just watched had no CLAUDE.md; with one, the agent would have known the test command before its first edit.

A tour of .claude/: small files that shape every session



TIP

/grill-me to figure out the foggy areas of an idea

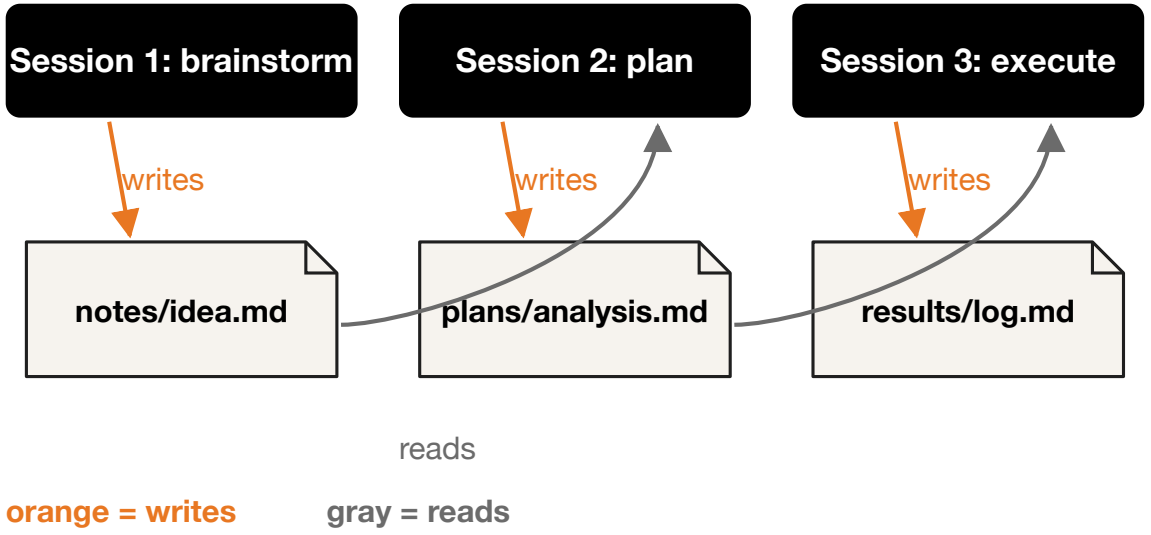
TIP

Providing it a clearly defined, high-level /goal and checking in on the agent after the agent completed

TIP

Write a pr.md that explains how to make PR for your project

Markdown is the agent's external memory



Treat a long task like a research article

question
method
results
open questions

TIP

write brainstorming and planning notes to markdown so you have a record and can use that for context in a future session

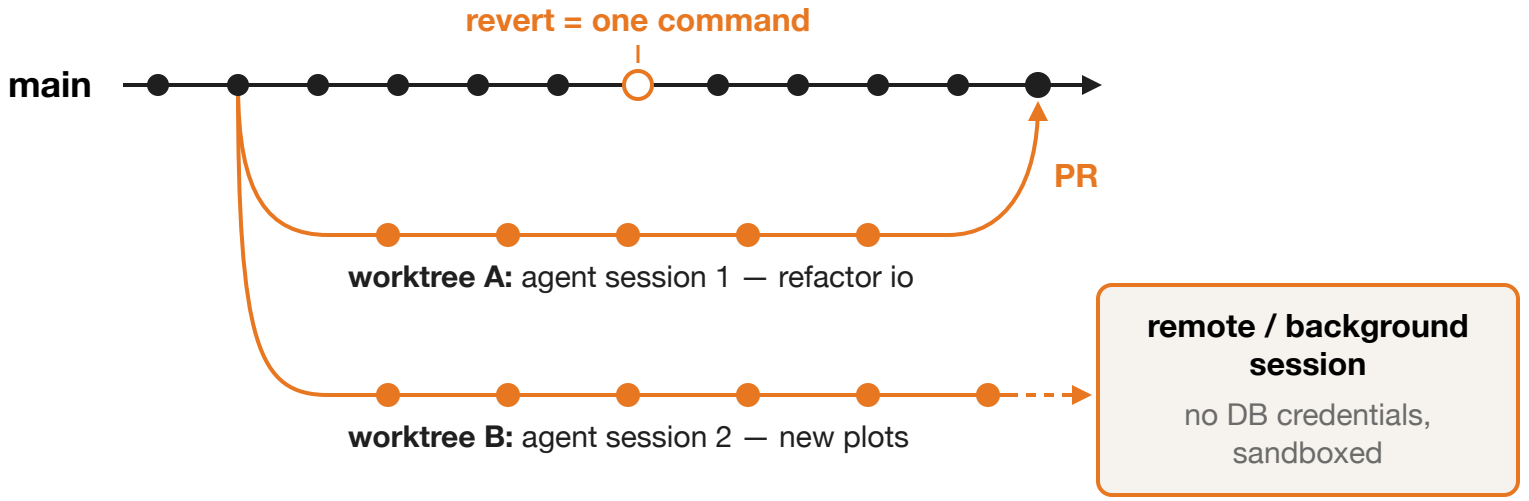
TIP

Using skills, treating a long task as a "research article" so it is easy to track and verify, using git as a way of tracking progress for bigger projects

You get a record; the next session gets context.

Git is the safety net

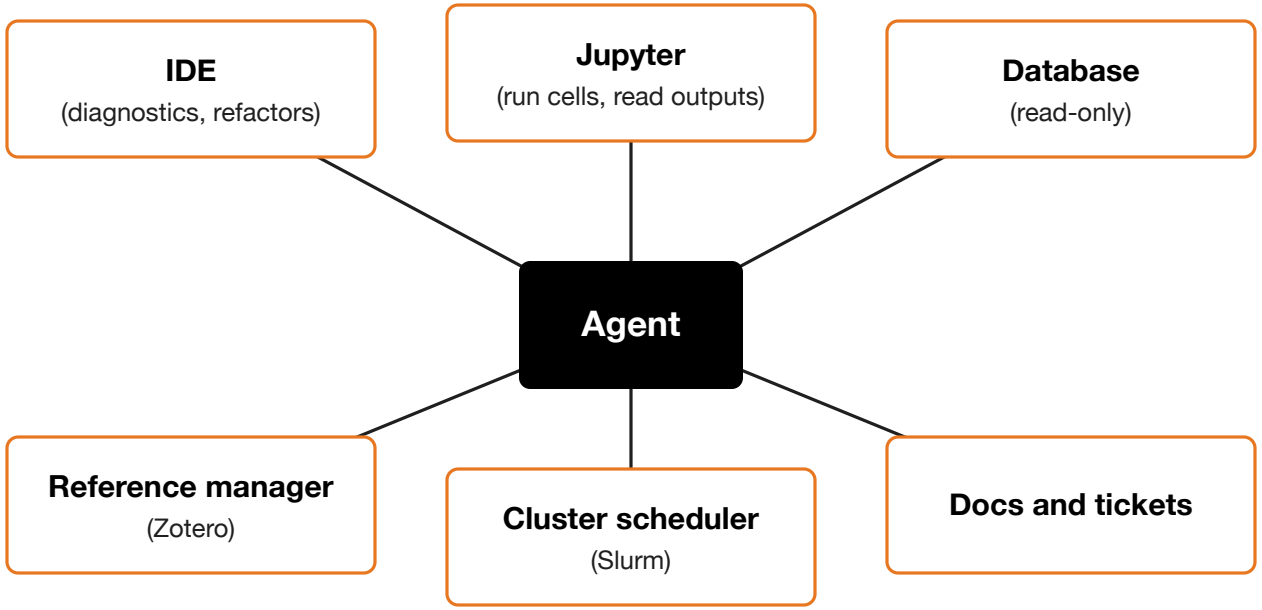
- **Commit early and often:** any agent change is one `git checkout` from undone.
- **Worktrees:** several agent sessions in parallel, each on its own branch of the same repo.
- **Long runs go remote:** background session, sandboxed, no database access.



TIP

git worktrees and a remote environment without DB access have long-running sessions operating in the background

MCP: connect the agent to what you already use



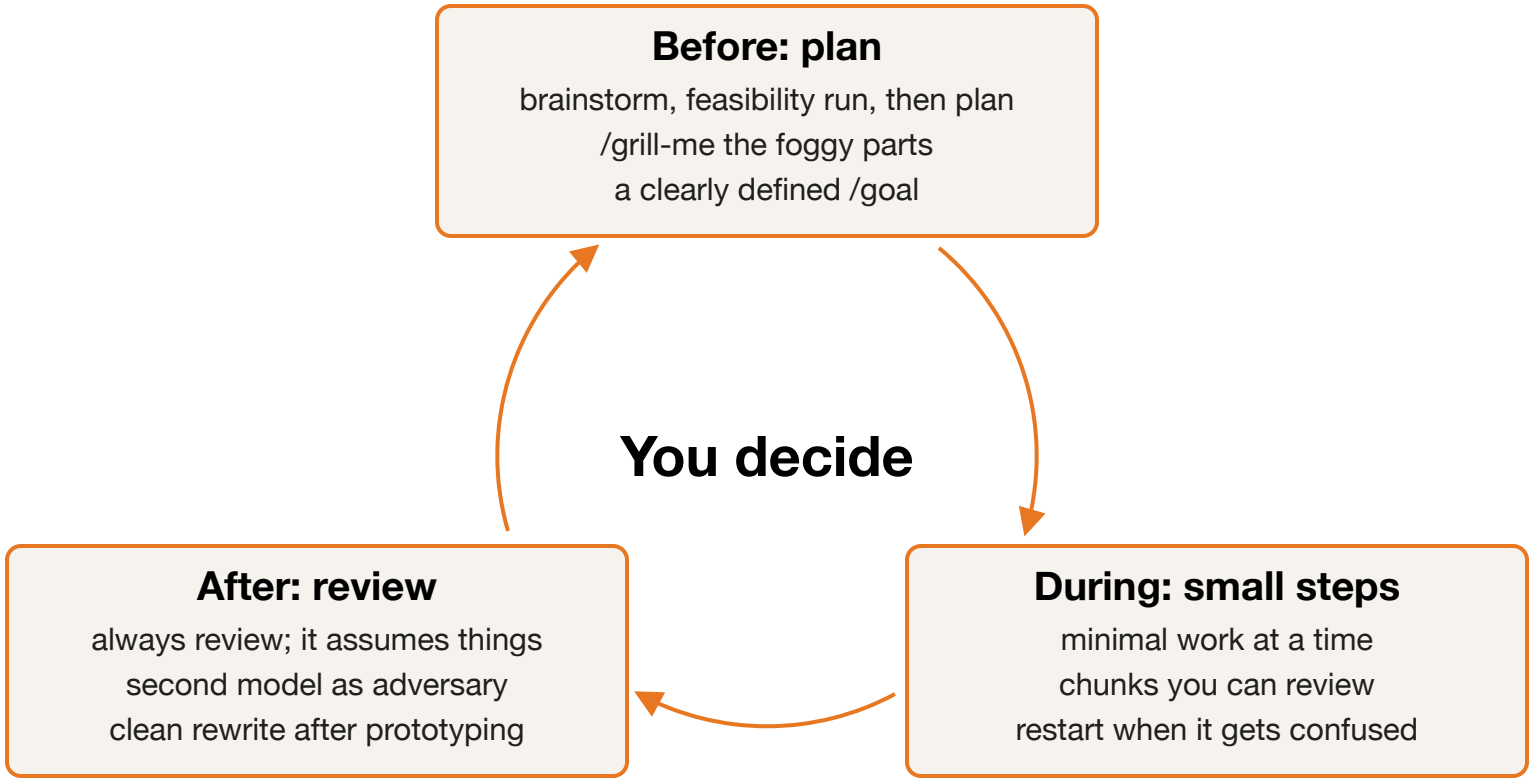
MCP = Model Context Protocol, one plug shape for all of them

TIP

Have MCP server running to utilize IDE features

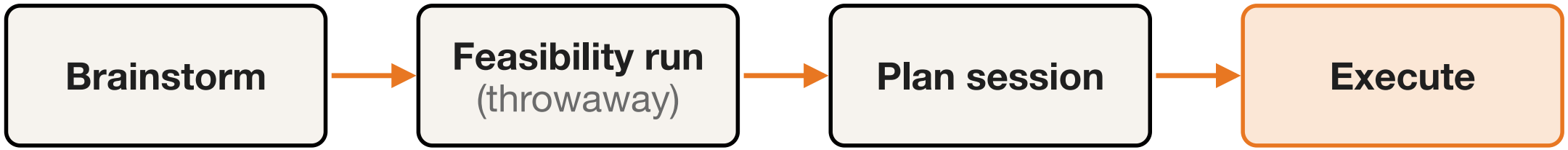
Examples only; no setup walkthrough today. Each server is a few lines in settings.

Supervising is a loop: plan, small steps, review



The next three slides are one lap around this loop. The tips are your colleagues', quoted as written.

Before: plan, and let it interrogate the plan



/grill-me finds the foggy parts

/goal: one clearly defined outcome, then check in when it is done

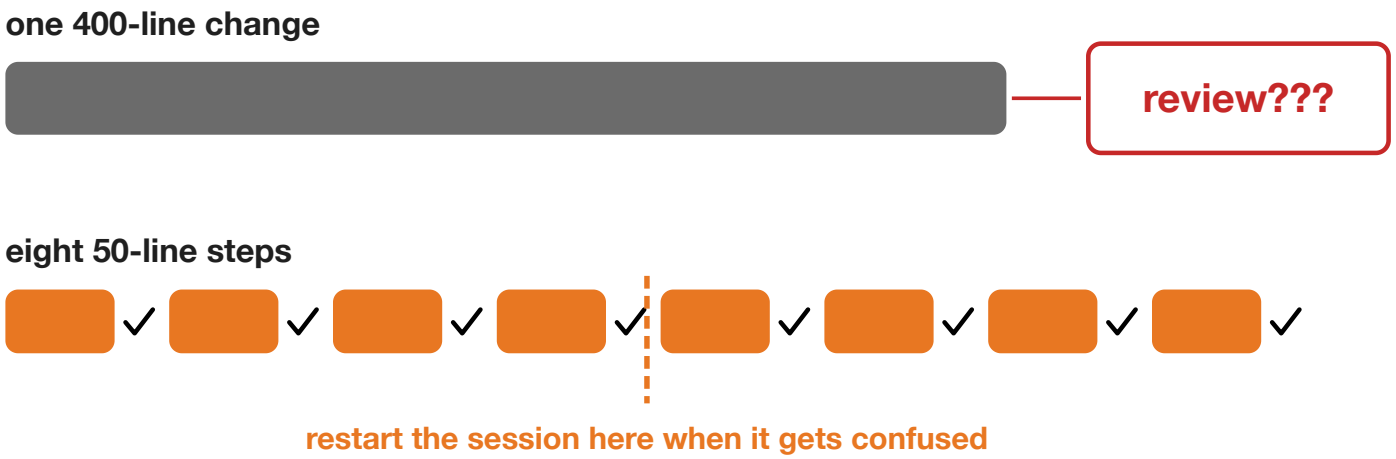
TIP
Brainstorm + feasibility run before running the plan session

TIP
/grill-me to figure out the foggy areas of an idea

TIP
Providing it a clearly defined, high-level /goal and checking in on the agent after the agent completed

Planning is cheaper than fixing: a bad plan costs a page of markdown, a bad build costs a day of review.

During: small steps you can actually review



TIP

work in small chunks so you can understand and review

TIP

Having it do minimal work at a time

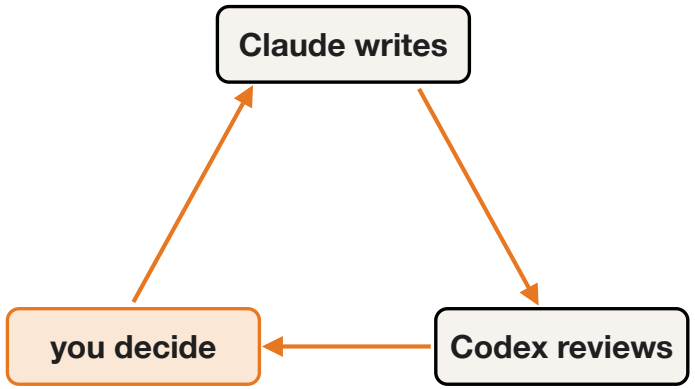
TIP

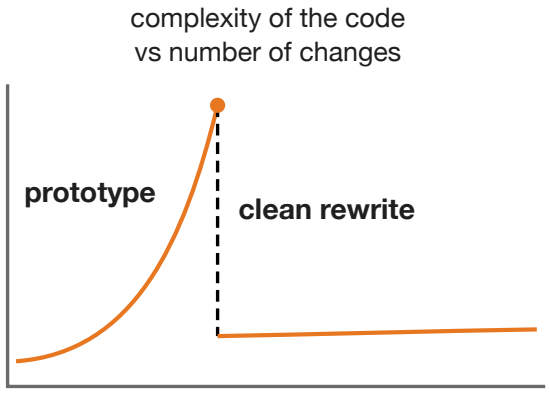
Don't get stuck with too much AI written code that then needs to be reviewed

TIP

Have the AI write code to do the task, not directly do the task

After: review as if a student handed it in





- ✓ Read every line you would sign for
- ✓ Ask it to list the problems in its own code
- ✓ After prototyping, rewrite clean

TIP

Always review the code. It often makes mistakes or assumes things different than your implicit assumptions

TIP

it often creates an over engineered mess after a while of changes. Once the prototyping is done it is good to do a clean rewrite

Mindset: ambitious on the problem, strict on the output

Be ambitious.

TIP

be extremely ambitious, especially regarding attempting new research or mathematical approaches

Build artifacts to understand.

TIP

Generating Claude ai artifacts to understand complicated ideas

Write your own prose.

TIP

Don't use it to write prose. If you can't be bothered to write it then no-one should be bothered to read it

The tool lowers the cost of trying. It does not lower the bar for what you hand in.

Each failure mode has a counter-move

When it…	You…
Over-engineers after many changes	rewrite clean once the prototype works
Drifts from your assumptions silently	ask it to list its assumptions before it runs; state units, ranges, NaN policy in the prompt
Hallucinates a dependency or API	run it; pin versions; let a subagent check the docs
Says "tests pass" without running them	make it show the command and the output; run the suite yourself before the PR
Answers an unverifiable question confidently	ask for the check it would run, then run it

Pairs with slide 13; same failures, now the response.

TIP

it often creates an over engineered mess after a while of changes. Once the prototyping is done it is good to do a clean rewrite

One page to take home

Setup

- ✓ CLAUDE.md with layout, test command, conventions
- ✓ Git: commit often, worktrees for parallel sessions
- ✓ Permissions: sandbox, no live credentials

Loop

- ✓ Plan first, /grill-me the foggy parts
- ✓ Small steps you can review
- ✓ Review as a supervisor, second model as adversary

Responsibility

- ✓ Disclose per the project's or journal's policy
- ✓ Mind the data: nothing restricted into a hosted model
- ✓ Never let an agent open a PR you have not read

Clone the starter kit: `starter-kit/` in this talk's repo (CLAUDE.md skeleton, a /grill-me command, a test-runner agent, a permissions file).

Your commitment card

One task I will hand to an agent this week:

_____

How I will check the result:

_____

Same card tool as the tips. We read three aloud.

Discussion

1. What is one task in your research you would hand to an agent tomorrow, and how would you check the result?
2. Which of your data may not leave the cluster, and what would you run locally instead?
3. Iterate on the prototype, or rewrite clean: when does each win?

Parking-lot questions come back here.

Parking lot

Questions that ran over. We come back to these in the discussion.

Sources and thanks

- AI Policy, Disclosure, and Human in the Loop (Hora & Robbes, ICSME 2026)
- Linux kernel coding-assistants policy
- Phoronix, 13 Oct 2026, on the Assisted-by patch
- LLVM AI policy
- GCC AI policy
- RedMonk, generative AI policy landscape in open source
- Artificial Analysis Intelligence Index
- Vendor pricing pages and all numbers with access dates: research/numbers.md in this repo.

The tips in section 5 are quoted as written from 23 colleagues on campus who answered "What tricks have you found helpful when it comes to agentic AI?" Thanks to the Research Computing group for input on these slides and for many discussions on AI coding.